

EV Market Analysis

Complete Python Data Science Project
ev_market_2026.csv | Figures 5.2 – 7.2

This document contains the complete, commented Python code for the EV Market Data Science project. Sections 0-1 cover data loading and feature engineering; Sections 2-8 reproduce all exploratory figures; Sections 9-11 build the Random Forest price model; and Sections 12-14 implement the sales forecasting pipeline using Gradient Boosting Regressors.

Section 0 — Imports & Setup

Import all required libraries: pandas, numpy, matplotlib, seaborn, and sklearn.

```
"""
EV Market Data Science Project – Full Python Code
Reproduces all figures from the analysis.
"""
```

Section 1 — 1. LOAD DATA

Load the `ev_market_2026.csv` dataset and engineer derived features: price in thousands, efficiency score (miles/kWh), power-to-weight ratio, and value index.

[illegible]

Section 2 — TOP 10 BRANDS BY TOTAL SALES

Aggregate annual_sales_units by brand and plot a horizontal bar chart of the top 10 brands. Tesla is highlighted in red; all others in blue.

[illegible]

Section 3 — PRICE vs RANGE BY SEGMENT

Scatter plot of range_miles vs price (USD thousands) coloured by market segment (Budget, Mid-range, Premium, Luxury).

[illegible]

Section 4 — FEATURE CORRELATION HEATMAP

Compute a Pearson correlation matrix for 12 numeric features and render a colour-coded heatmap using seaborn. Red = strong positive, Blue = strong negative.

```
#
# 3. FIG 5.3 – PRICE vs RANGE BY SEGMENT
#
seg_colors = {"Budget": "#2ecc71", "Mid-range": "#3498db",
              "Premium": "#f39c12", "Luxury": "#e74c3c"}

fig, ax = plt.subplots(figsize=(11, 6))
for seg, grp in df.groupby("market_segment"):
    ax.scatter(grp["range_miles"], grp["price_usd_k"],
               color=seg_colors.get(seg, "grey"), alpha=0.75, s=60, label=seg)
ax.set_xlabel("Range (miles)")
ax.set_ylabel("Price (USD thousands)")
ax.set_title("Fig 5.3 – Price vs Range by Market Segment", fontweight="bold", pad=12)
ax.legend(title="Segment")
plt.tight_layout()
plt.savefig("fig5_3_price_vs_range.png", dpi=150)
plt.close()
```

Section 5 — MARKET SEGMENT SHARE (COUNT & SALES)

Side-by-side pie charts showing segment share by vehicle count (left) and by total sales volume (right).

```
#
# 4. FIG 5.4 – FEATURE CORRELATION HEATMAP
#
num_cols = ["price_usd_k", "battery_capacity_kwh", "range_miles",
            "charging_speed_kw", "acceleration_0_60_mph", "horsepower",
            "safety_rating", "annual_sales_units", "customer_rating",
            "efficiency_score", "power_to_weight", "value_idx"]

corr = df[num_cols].rename(columns={
    "price_usd_k": "price_usd", "battery_capacity_kwh": "battery_kwh",
    "charging_speed_kw": "charging_kw", "acceleration_0_60_mph": "accel_060",
    "safety_rating": "safety", "annual_sales_units": "annual_sales",
    "customer_rating": "cust_rating", "power_to_weight": "power_wt"
}).corr()

fig, ax = plt.subplots(figsize=(12, 9))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r",
            center=0, linewidths=0.4, ax=ax,
            annot_kws={"size": 8})
ax.set_title("Fig 5.4 – Feature Correlation Heatmap", fontweight="bold", pad=12)
plt.tight_layout()
plt.savefig("fig5_4_heatmap.png", dpi=150)
plt.close()
```

Section 6 — EV SALES EVOLUTION BY COUNTRY

Line chart of aggregate annual_sales_units per year for the top 5 countries of origin: US, Germany, South Korea, China, and Japan.

[illegible]

Section 7 — BATTERY CAPACITY vs EFFICIENCY

Scatter plot of battery_capacity_kwh vs efficiency_score coloured by price, using the YlOrRd colormap to convey price gradient.

[illegible]


```
model_labels = top12_models["brand"] + " " + top12_models["model"]

fig, ax = plt.subplots(figsize=(10, 7))
bars = ax.barh(model_labels, top12_models["annual_sales_units"] / 1e6, color=model_colors)
for bar, val in zip(bars, top12_models["annual_sales_units"] / 1e6):
    ax.text(bar.get_width() + 0.1, bar.get_y() + bar.get_height() / 2,
            f"{val:.0f}M", va="center", fontsize=8, fontweight="bold")
ax.set_xlabel("Sales (Millions)")
ax.set_title("Fig 5.8 – Top 12 EV Models by Total Sales", fontweight="bold", pad=12)
from matplotlib.patches import Patch
legend_elements = [Patch(fc=c, label=b) for b, c in brand_color_map.items()]
ax.legend(handles=legend_elements, loc="lower right")
plt.tight_layout()
plt.savefig("fig5_8_top12_models.png", dpi=150)
plt.close()
```

Section 10 — RANDOM FOREST: ACTUAL vs PREDICTED + RESIDUALS

Train a RandomForestRegressor (200 trees), evaluate on the test set, and produce two subplots: actual vs predicted price scatter, and residual distribution histogram.

[illegible]

Section 11 — TOP 15 FEATURE IMPORTANCES

Extract feature importances from the trained Random Forest and plot the top 15 as a horizontal bar chart. Top 2 features (horsepower, market segment enc) are red.

```
# ██████████  
# 10. FIG 6.2 – RANDOM FOREST: ACTUAL vs PREDICTED + RESIDUALS  
# ██████████  
rf = RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1)  
rf.fit(X_train, y_train)  
y_pred = rf.predict(X_test)  
residuals = y_test.values - y_pred  
  
mae = mean_absolute_error(y_test, y_pred)  
r2 = r2_score(y_test, y_pred)  
print(f"Random Forest → MAE: {mae:.2f} K | R²: {r2:.4f}")  
  
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
```

```
# Actual vs Predicted
ax = axes[0]
ax.scatter(y_test, y_pred, alpha=0.5, color="#4a90d9", s=25)
lims = [min(y_test.min(), y_pred.min()), max(y_test.max(), y_pred.max())]
ax.plot(lims, lims, "r--", linewidth=1.5, label="Perfect fit")
ax.set_xlabel("Actual (USD K)")
ax.set_ylabel("Predicted (USD K)")
ax.set_title("Actual vs Predicted Price\n(Random Forest)", fontweight="bold")
ax.legend()

# Residual distribution
ax2 = axes[1]
ax2.hist(residuals, bins=30, color="#4a90d9", edgecolor="white")
ax2.axvline(0, color="red", linestyle="--", linewidth=1.5)
ax2.set_xlabel("Residual (USD K)")
ax2.set_ylabel("Count")
ax2.set_title("Residual Distribution", fontweight="bold")

fig.suptitle("Fig 6.2 – Actual vs Predicted & Residual Distribution",
             fontweight="bold", y=1.01)
plt.tight_layout()
plt.savefig("fig6_2_rf_residuals.png", dpi=150, bbox_inches="tight")
plt.close()
```

Bar chart combining historical data (blue bars, 2020-2026) with simple trend-based forecasts (red bars, 2027-2031). A trend line connects all data points.

For each of the top 5 brands, fit a GradientBoostingRegressor on historical yearly sales and forecast through 2031. Historical = solid lines; Forecast = dashed lines.

```
ax.text(yr, val + 0.5, f"{val}M", ha="center", fontsize=8, fontweight="bold")
ax.plot(hist_years + fore_years, hist_sales + fore_sales,
        "o-", color="black", linewidth=1.2, markersize=4)
ax.set_xlabel("Year")
ax.set_ylabel("Sales (Millions)")
ax.set_title("Fig 7.1 – Global EV Sales Forecast 2027-2031", fontweight="bold", pad=12)
ax.legend()
plt.tight_layout()
plt.savefig("fig7_1_global_forecast.png", dpi=150)
plt.close()
```

Global-level GBR model trained on 2020-2025 full-year totals. Produces a stacked bar + trend-line chart distinguishing observed Q1 2026 data from the annualised projection and 2027-2031 forecast.